

FlowTO Traffic Digital Twin

How the simulation backend, API, and interactive planning frontend work together.

READING TIME

About 4 minutes. This guide is based on the current repository implementation rather than the older planning specifications.

Overview

FlowTO is a Toronto traffic and transit planning tool. It lets planners load a city road network, view congestion, test road closures, and compare intervention results. The showcase scenario models FIFA World Cup post-match traffic around BMO Field.

The backend is a FastAPI service around a Python simulation engine. The frontend is a React, Vite, and TypeScript app using deck.gl with a Mapbox basemap. The system is local-first: the CPU simulation is the reliable demo path, while GPU and local AI features are optional DGX Spark upgrades.

Architecture Diagram

DATAFLOW

Toronto open data -> baked graph and demand model -> simulation engine -> FastAPI REST and WebSocket API -> Zustand state and typed arrays -> deck.gl map and planning panels.

At A Glance

Road Network

A stable edge index and geometry table is loaded once, then reused for repainting.

Simulation

BPR congestion, equilibrium tooling, and blast-radius recompute power scenario runs.

Planner UI

Simulate and Edit modes expose the map, interventions, saved scenarios, and metrics.

Local-First Stack

No runtime database or queue is required. Optional Spark services improve the demo.

Backend & API

The FastAPI application loads a read-only baseline graph and demand-derived origin-destination matrix at startup. Scenario state is held in memory. JSON snapshots can be enabled when the API is created with a snapshot directory.

GET /healthz

Service status and graph size.

GET /demo/run?scenario=...

Cached **baseline**, **wc_surge**, or **wc_fix** demo output.

PATCH /scenarios/{id}

Update name, weather, time context, or interventions.

GET /scenarios/{id}/records

Return per-edge results for a map repaint.

POST /scenarios/{id}/preview

Run without mutating the stored scenario.

GET /transit/routes

Return cached real or fallback TTC route lines.

WS /scenarios/{id}/stream

Send compact binary tick frames for live scenarios.

GET /edges

Stable edge index, geometry, road name, and class.

POST /scenarios

Create an editable scenario.

POST /scenarios/{id}/run

Run a full or blast-radius recompute.

GET /scenarios/{id}/compare

Return before/after metric deltas.

POST /copilot/plan

Return a preview-first intervention plan with citations.

GET /transit/trajectories

Return cached real or fallback vehicle trajectories.

POST /optimize

Propose a heuristic, simulation-verified improvement.

AUTHENTICATION

Authentication and multi-tenant access are intentionally out of scope. Development CORS allows all origins so the Vite frontend can call the API locally.

EXAMPLE SCENARIO

```
{
  "name": "Close Lake Shore",
  "interventions": [
    {"op": "close_edge", "edge_id": "12345"}
  ]
}
```

EXAMPLE RUN

```
{
  "recompute": "blast",
  "congestion_model": "bpr"
}
```

MAP RECORD

```
[edge_index, load, speed_kmh,
 pressure, closure_flag]
```

Frontend

- `App.tsx` assembles the map, docks, status bar, edit tools, and first-run screen.
- `MapCanvas.tsx` renders roads, transit lines, closures, pins, and the BMO Field marker.
- `appStore.ts` handles loading, saved scenarios, interventions, recomputing, and copilot requests.
- `tickStore.ts` keeps pressure updates in typed arrays outside React for fast repainting.
- `api/client.ts` uses REST `fetch()` calls and supports the binary WebSocket stream.
- Vite proxies `/api` to `http://localhost:8000` during development.

PRIMARY UI FLOW

Load GET `/edges`, then load GET `/demo/run?scenario=baseline`. In Edit mode, create or patch a scenario, run a blast-radius recompute, fetch records, and repaint the deck.gl road layer.

Integration Points

Open Data

Toronto CKAN datasets, GTFS transit feeds, and weather or restriction inputs.

Offline Data Store

Parquet files and a DuckDB catalog support reproducible data preparation.

Map Rendering

Mapbox Standard basemap via `VITE_MAPBOX_TOKEN`, with deck.gl overlays.

Optional Spark Services

Ollama Nemotron, cuGraph GPU routing, and a self-hosted cuOpt endpoint.

Key Files & Folders

`src/torontosim/api/app.py`

FastAPI routes and WebSocket.

`src/torontosim/api/encoding.py`

Compact binary frame format.

`src/torontosim/datapipeline/`

Fetch, bake, catalog, and manifests.

`frontend/src/api/client.ts`

Typed frontend API client.

`frontend/src/components/MapCanvas.tsx`

Live map rendering.

`src/torontosim/api/store.py`

Scenario state and orchestration.

`src/torontosim/simulation/`

Traffic assignment and congestion engine.

`src/torontosim/copilot/`

Policy-aware intervention planner.

`frontend/src/state/appStore.ts`

Planner workflow state.

`scripts/run_api.sh`

Local FastAPI launcher.

Quick Troubleshooting

Map is blank.

Set `VITE_MAPBOX_TOKEN` in `frontend/.env`. The viewport shows a token-required message when Mapbox Standard cannot initialize.

Frontend cannot load data.

Run `scripts/run_api.sh`, then check `GET /healthz`. The frontend reports an API connection error if the baseline graph cannot be reached.

Demand surge edits do not affect results.

The frontend can place a demand-surge object, but backend schema support is still pending. Road closures are the complete end-to-end edit path.

Technical Appendix

Runtime State

- No runtime database or queue is required.
- Scenarios are in-memory dictionaries keyed by short generated IDs.
- The baseline graph and edge-index mapping are shared across scenario runs.

Streaming Contract

- Binary frames are little-endian and start with a u32 record count.
- Each 17-byte record contains `edge_idx:u32`, three `f32` values, and `closure:u8`.
- The main showcase uses REST demo records; the client also supports WebSocket ingestion.

Fallback Strategy

- Default graph: committed OSMnx JSON. Optional graph: baked Centreline Parquet.
- Default transit: hand-authored demo routes. Optional transit: cached real GTFS feeds.
- Default compute: CPU. Optional compute: cuGraph GPU backend on DGX Spark.

Run Locally

```
# Backend
scripts/run_api.sh

# Frontend
cd frontend && npm install && npm run dev
```

API: <http://localhost:8000> | Frontend: <http://localhost:5173> | Status: **CPU demo path available**